

Setting Up the ESP32-IDF Development Environment

Setting Up the ESP32-IDF Development Environment

1. [ESP32 Development Environment Introduction](#)
2. [Installing Dependencies](#)
3. [Downloading ESP-IDF](#)
4. [Activating the ESP-IDF Development Environment](#)
5. [Compiling and Flashing the Firmware](#)
6. [Quick Commands](#)
7. [ESP-IDF API Reference](#)

Note: The out-of-the-box virtual machine already includes the environment, so there is no need to download it again.

1. ESP32 Development Environment Introduction

The ESP32 development environment supports Windows, Linux, and Mac platforms.

On the Linux and Mac platforms, you need to download and install the ESP-IDF development environment from source.

On the Windows platform, there are multiple development environment options available, including the official ESP-IDF environment, development environments based on the Arduino IDE, and development approaches using VS Code + PlatformIO IDE, etc.

Here, we take the approach of downloading and installing the ESP-IDF development environment from source on an Ubuntu system as an example.

2. Installing Dependencies

Open an Ubuntu system terminal and run the following command to install the required dependencies.

```
sudo apt-get install git wget flex bison gperf python3 python3-pip python3-venv  
cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

3. Downloading ESP-IDF

Note: The out-of-the-box virtual machine already includes the environment, so there is no need to download it again.

Open an Ubuntu system terminal and run the following command to download esp-idf version v5.1.2.

```
mkdir -p ~/esp
cd ~/esp
git clone -b v5.1.2 --recursive https://github.com/espressif/esp-idf.git
```

Set the chip supported by the tools to esp32s3.

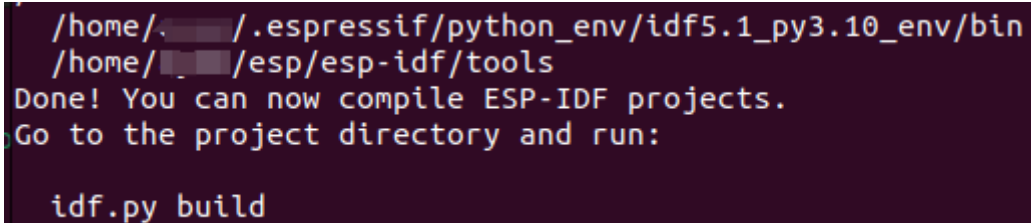
```
cd esp-idf
./install.sh esp32s3
```

4. Activating the ESP-IDF Development Environment

Run the following command in the esp-idf tools directory.

```
source ~/esp/esp-idf/export.sh
```

Note: Every time you open a new terminal, you must first activate the ESP-IDF development environment before you can compile ESP-IDF projects.



```
/home/.../.espressif/python_env/idf5.1_py3.10_env/bin
/home/.../esp/esp-idf/tools
Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

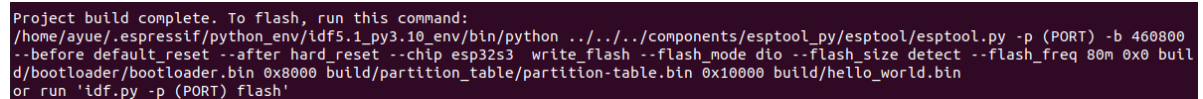
idf.py build
```

5. Compiling and Flashing the Firmware

Connect the microROS control board to the virtual machine/computer, and then test compiling the hello_world program to generate the firmware.

```
cd examples/get-started/hello_world
idf.py set-target esp32s3
idf.py build
```

When you see the following prompt, it means the compilation succeeded.



```
Project build complete. To flash, run this command:
/home/ayue/.espressif/python_env/idf5.1_py3.10_env/bin/python ../../components/esptool_py/esptool/esptool.py -p (PORT) -b 460800
--before default_reset --after hard_reset --chip esp32s3 write_flash --flash_mode dio --flash_size detect --flash_freq 80m 0x0 build/
bootloader/bootloader.bin 0x8000 build/partition_table/partition-table.bin 0x10000 build/hello_world.bin
or run 'idf.py -p (PORT) flash'
```

View the generated firmware.

```
ls build/*.bin
ls build/bootloader/*.bin
ls build/partition_table/*.bin
```

```
~/esp/esp-idf/examples/get-started/hello_world$ ls build/*.bin
build/hello_world.bin
~/esp/esp-idf/examples/get-started/hello_world$ ls build/bootloader/*.bin
build/bootloader/bootloader.bin
~/esp/esp-idf/examples/get-started/hello_world$ ls build/partition_table/*.bin
build/partition_table/partition-table.bin
~/esp/esp-idf/examples/get-started/hello_world$
```

Run the following command to flash the generated firmware to the microROS control board.

```
idf.py flash
```

Before flashing with this command, you need to confirm that the computer is connected to only one serial device. If multiple serial devices are connected, you can manually specify the serial port number to distinguish them. For example, specify the serial port for flashing as `/dev/ttyUSB0`.

```
idf.py flash -p /dev/ttyUSB0
```

When the download progress reaches 100% and "Done" is displayed, the flashing is complete.

```
Writing at 0x00028d92... (57 %)
Writing at 0x0002f43f... (71 %)
Writing at 0x00037267... (85 %)
Writing at 0x0003cd96... (100 %)
Wrote 202224 bytes (109806 compressed) at 0x00010000 in 2.8 seconds
Hash of data verified.
Compressed 3072 bytes to 103...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (103 compressed) at 0x00008000 in 0.1 seconds (effective 1.5MB/s)
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
Done
```

Open the serial monitor.

```
idf.py monitor
```

To exit the serial monitor, press **Ctrl+J**.

6. Quick Commands

If you need to compile, flash, and open the serial monitor, enter the following command.

```
idf.py build flash monitor
```

7. ESP-IDF API Reference

Open the following link to view the official ESP32S3 API reference.

```
https://docs.espressif.com/projects/esp-idf/en/v5.1.2/esp32s3/api-reference/index.html
```

If you need to display Chinese, you can click **【中文】** to go to the Chinese version.

API Reference

[\[中文\]](#)

